DSA 495 601 Project Milestone 4

Thomas Grover

- Set-up
- Step 1: Specify the prediction objective
- Step 2: Specify outcome variable name
- Step 3: Specify predictor sets
- Step 4: Specify modeling approaches
- Step 5: Create recipes for tidymodels
- Step 6: Specify cross-validation parameters
- Step 7: Specify variables for fairness assessment
- Step 8: Review and verify specifications
- Step 9: Aggregate and save all specifications

For Milestone 4, you will specify your learner configurations and prepare for model training. This R notebook provides guidance on specifying your prediction objective, predictor sets, modeling approaches, cross-validation parameters, and fairness assessment variables. Please read through the entire document closely.

Your goals are to (1) clearly define your prediction objective, (2) specify one or more predictor sets to test, (3) select appropriate modeling approaches for each predictor set, (4) configure cross-validation parameters, and (5) identify variables for fairness assessment.

To help you identify places where you need to edit the code to match your project, look for “@TODO” in comments. Please delete examples discussed in the text and add your own explanations for your dataset and project goals. Answer all questions indicated with “@TODO”.

After you have edited the code as needed for your project and run all code chunks successfully, knit your completed file to HTML. Download the HTML to your computer and then print or save it to a PDF file with the following naming convention: course_section_semester_assignment_unityID.pdf. For example, DSC495_601_FA25_Milestone4_keporte2.pdf.

Set-up

Load the necessary packages

No edits are needed in the following chunk. Just run it as is.

```
.libPaths(c("/share/dsa495601f25/installed_libraries", .libPaths()))
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyr)
library(recipes)
```

```
##
## Attaching package: 'recipes'
```

```
## The following object is masked from 'package:stats':
##
##     step
```

```
# Set seed for reproducibility
seed <- 2022

# Source check files
# source(paste0(params$my_path,"R/","checks_01_03.R"))
# source(paste0(params$my_path,"R/","checks_helpers.R"))
```

Load your processed data files

Load the training and testing data files you created and saved in Milestone 3.

```
# Load processed training and testing data from Milestone 3
trainDat <- readRDS(paste0(params$my_path, "trainDat.rds"))
testDat  <- readRDS(paste0(params$my_path, "testDat.rds"))

# Check that files loaded correctly
dim(trainDat)
```

```
## [1] 31647    73
```

```
dim(testDat)
```

```
## [1] 13564    73
```

Step 1: Specify the prediction objective

Provide clear, concise descriptions of your prediction objective. These descriptions will help document your project and ensure clarity about what you're predicting, when, and for whom.

Question 1: Describe your outcome, timepoint, and population in 1-2 sentences each. This is a review of your scoping, or it may be an update of your scoping if I gave your feedback that suggested changes.

The outcome is the variable y, which represents whether or not a customer will open a term deposit account with the bank through the marketing campaign The timepoint is any time throughout the duration of the marketing campaign. The population is all the customers contacted throughout the campaign.

```
# Provide short text descriptions for outcome, timepoint, and population
outcomeDescription <- "I am predicting whether or not a customer will open a term deposit accoun
t"
timepointDescription  <- "I am making predictions throughout the entire duration of the marketin
g campaign."
populationDescription <- "The population is all of the customers contacted as a part of a market
ing campaign."
```

Step 2: Specify outcome variable name

Enter the name of the column in your training and testing datasets that contains your outcome variable.

Reminder: Your outcome variable should be binary (taking values 0 and 1) with no missing values, as created in Milestone 3.

```
# Specify the name of your outcome variable column
outcomeName <- "y_binary"
```

Step 3: Specify predictor sets

In this step, you will define one or more sets of predictor variables to test in your models. Each predictor set should be named using the convention `predSet_` followed by a short descriptive name.

Important notes:

- Your predictor sets should only include the processed variables from Milestone 3 (scaled versions, dummy coded variables, missing indicators, etc.)
- Do NOT include the original unprocessed variables unless they did not need processing (they are in the correct format and have no missing values)
- For every predictor with a missing indicator variable created in Milestone 3, include both the imputed variable AND the missing indicator
- If you have multiple dummies for categorical variables, include all EXCEPT ONE in your predictor set. This avoids perfect multicollinearity - when all dummy variables are included, they sum to 1, which means any one can be perfectly predicted from the others. The excluded category becomes the “reference group” that other categories are compared against.
- Start with a **benchmark** predictor set that includes only essential baseline predictors

Question 2: Describe your benchmark predictor set. What variables did you include and why? What timepoint(s) are these variables available?

For the benchmark predictor set I includes 5 of the standardized core numerical values from the bank-marketing dataset. These variables represent key numerical information about the demographic, financial situation, and history of communication of the customer. I chose these for the benchmark because they represent important predictive information about a client without introducing engineered or more complex variables.

```
# Specify your benchmark predictor set
# Include processed variable names from Milestone 3
predSet_bm <- c("age_std",
                "balance_std",
                "duration_std",
                "campaign_std",
                "previous_std")   # Handled pdays = -1 ("Never Contacted" indicator) in Mileston
e 3 by filtering for only non -1 observations

# Print to verify
print(predSet_bm)
```

```
## [1] "age_std"      "balance_std"  "duration_std" "campaign_std" "previous_std"
```

Next, specify at least 2-3 additional predictor sets. Your strategy is to incrementally add more features (variables) that may also be predictive. Your final predictor set may be a “kitchen sink” set, taking advantage of a large set of variables that may or may not be predictive (maybe everything availalbe but not necessarily).

Question 3: For additional predictor sets beyond the benchmark, describe each one. What makes it different from the benchmark set? What hypothesis are you testing with this predictor set?

The first extended predictor set is designed to test how contact type, and contact history can impact the outcome. I included the engineered variables “long_call” and “previously_contacted”, as well as 2/3 binary categories for contact type. These combine to give us valuable information about contact history and engagement with a customer. This set is essentially testing the hypothesis that more engagement (e.g. previous calls, longer calls) increases the conversion rate. It is different than the benchmark because it includes some of the engineered and one-hot encoded variables from Milestone 3.

The second extended predictor set is designed to test how financial situation, and the liquidity of a customer affects the outcome. I included 2 engineered variables being “negative_balance” and “high_balance” to get an idea of their financial status. I also included one of the loan status variables because that is another aspect that can affect financial decision making, if someone has to make loan payments they are probably less likely to open an account. This set is designed to test the hypothesis that more liquidity and fewer financial obligations are more likely to invest in a term deposit.

Lastly, we have our kitchensink which incorporates every variable we have engineered, dummy-coded, and standardized. The goal is to determine which of the variables have the greatest predictive power by including them all in one model.

```
# EXAMPLE 1: Exploring if how someone is contacted and contact duration are predictive
predSet_extended1 <- unique(c(predSet_bm,
                              c("long_call",
                                "previously_contacted",
                                "contact_cellular",
                                "contact_telephone")))    # Have to leave out contact_unknown be
cause you must use k-1 of the categories of a one-hot encoded var
print(predSet_extended1)
```

```
## [1] "age_std"              "balance_std"          "duration_std"
## [4] "campaign_std"         "previous_std"         "long_call"
## [7] "previously_contacted" "contact_cellular"     "contact_telephone"
```

```
# EXAMPLE 2: Exploring if account balance / liquidity is predictive
predSet_extended2 <- unique(c(predSet_bm,
                              c("negative_balance",
                                "high_balance",
                                "loan_yes")))
print(predSet_extended2)
```

```
## [1] "age_std"          "balance_std"      "duration_std"     "campaign_std"
## [5] "previous_std"     "negative_balance" "high_balance"     "loan_yes"
```

```
# EXAMPLE 3: Including all features (Kitchen Sink)
predSet_kitchensink <- c(predSet_bm,

                         #Engineered Features
                         "negative_balance",
                         "high_balance",
                         "previously_contacted",
                         "long_call",

                         # Contact Type (One-Hot Encoded, k-1 categories included)
                         "contact_cellular",
                         "contact_telephone",

                         # Loan Status (One-Hot Encoded, k-1 categories included)
                         "loan_yes",

                         # Housing Status (One-Hot Encoded, k-1 categories included)
                         "housing_yes",

                         # Default Status (One-Hot Encoded, k-1 categories included)
                         "default_yes",

                         # Education Level (One-Hot Encoded, k-1 categories included)
                         "education_primary",
                         "education_secondary",
                         "education_tertiary",

                         # Marital Status (One-Hot Encoded, k-1 categories included)
                         "marital_married",
                         "marital_single",

                         # Job Category (One-Hot Encoded, k-1 categories included)
                         "job_admin.",
                         "job_blue-collar",
                         "job_entrepreneur",
                         "job_housemaid",
                         "job_management",
                         "job_retired",
                         "job_self-employed",
                         "job_services",
                         "job_student",
                         "job_technician",
                         "job_unemployed",

                         # Month (One-Hot Encoded, k-1 categories included)
                         "month_jan",
                         "month_feb",
                         "month_mar",
                         "month_apr",
                         "month_may",
                         "month_jun",
                         "month_jul",
```

```
                          "month_aug",
                          "month_sep",
                          "month_oct",
                          "month_nov",

                          # Previous Outcome (One-Hot Encoded, k-1 categories included)
                          "poutcome_success",
                          "poutcome_failure",
                          "poutcome_other"

                          )
print(predSet_kitchensink)
```

```
##  [1] "age_std"              "balance_std"          "duration_std"
##  [4] "campaign_std"         "previous_std"         "negative_balance"
##  [7] "high_balance"         "previously_contacted" "long_call"
## [10] "contact_cellular"     "contact_telephone"    "loan_yes"
## [13] "housing_yes"          "default_yes"          "education_primary"
## [16] "education_secondary"  "education_tertiary"   "marital_married"
## [19] "marital_single"       "job_admin."           "job_blue-collar"
## [22] "job_entrepreneur"     "job_housemaid"        "job_management"
## [25] "job_retired"          "job_self-employed"    "job_services"
## [28] "job_student"          "job_technician"       "job_unemployed"
## [31] "month_jan"            "month_feb"            "month_mar"
## [34] "month_apr"            "month_may"            "month_jun"
## [37] "month_jul"            "month_aug"            "month_sep"
## [40] "month_oct"            "month_nov"            "poutcome_success"
## [43] "poutcome_failure"     "poutcome_other"
```

Now aggregate all your predictor sets into a single list:

```
# Creating a list of our 4 predictor sets to call upon in the future
allPredsets <- list(
  predSet_bm = predSet_bm,
  predSet_extended1 = predSet_extended1,
  predSet_extended2 = predSet_extended2,
  predSet_kitchensink = predSet_kitchensink
)

# Verify all predictor sets
print(names(allPredsets))
```

```
## [1] "predSet_bm"          "predSet_extended1"   "predSet_extended2"
## [4] "predSet_kitchensink"
```

Step 4: Specify modeling approaches

In this step, you will select which machine learning algorithms to use with each predictor set. Different algorithms make different assumptions and may perform better or worse depending on your data characteristics.

Available modeling approaches include:

- **glm**: Logistic regression (good baseline, interpretable)
- **lasso**: Regularized regression with automatic feature selection
- **ridge**: Regularized regression that keeps all features
- **elastic_net**: Combines lasso and ridge approaches
- **svm_linear**: Support vector machine with linear kernel
- **svm_rbf**: Support vector machine with radial basis function kernel
- **random_forest**: Ensemble of decision trees (handles non-linearity well)
- **xgboost**: Gradient boosted trees (often high-performing)

Considerations for choosing models:

- Start simple (e.g., glm) before trying complex models
- If you have many predictors, lasso can help with feature selection
- Tree-based methods (random_forest, xgboost) handle non-linear relationships well
- Review your notes and our slide deck from Class 7!

Question 4: For each predictor set, which modeling approaches did you choose? Why did you select these particular algorithms? Consider your dataset size, number of predictors, and project goals.

For the benchmark set, I used the glm and lasso approaches because the benchmark is smaller, and cleaner than all of the other predictor sets. Using simple and interpretable regression based models will get exactly what we want from our benchmark predictor set, a benchmark to compare other models to.

For all of the other 3 sets, I also used GLM and Lasso for comparison and interpretibility. I am also using the more complex tree based approaches. These predictor sets contain more variables, and likely more subtle interactions between them. I expect their to be some non-linear relationships between variables like contact type and duration, or loan status and balance. The tree based models are great at gaining insight into these interactions. I included glm and lasso with these datasets as well to get a baseline before jumping into the tree based models. I will use GLM first, compare with Lasso afterward, and then consider the tree based models.

```
# Specify which modeling approaches to use with each predictor set
specifiedLearners <- list(
  predSet_bm = c("glm", "lasso"),
  predSet_extended1 = c("glm", "lasso", "random_forest", "xgboost"),
  predSet_extended2 = c("glm", "lasso", "random_forest", "xgboost"),
  predSet_kitchensink = c("glm", "lasso", "random_forest", "xgboost")
)

# Verify specifications
print(specifiedLearners)
```

```
## $predSet_bm
## [1] "glm"   "lasso"
##
## $predSet_extended1
## [1] "glm"           "lasso"         "random_forest" "xgboost"
##
## $predSet_extended2
## [1] "glm"           "lasso"         "random_forest" "xgboost"
##
## $predSet_kitchensink
## [1] "glm"           "lasso"         "random_forest" "xgboost"
```

Step 5: Create recipes for tidymodels

The tidymodels package requires “recipes” to prepare data for modeling. Since you already prepared your data extensively in Milestone 3, these recipes are minimal - they mainly ensure your outcome variable is formatted correctly.

Note: You must create one recipe for each predictor set you specified above.

```r
# Create formulas for each predictor set
predSet_bm_formula <- as.formula(
  paste(outcomeName, "~", paste(predSet_bm, collapse = "+"))
)

predSet_extended1_formula <- as.formula(
  paste(outcomeName, "~", paste(predSet_extended1, collapse = "+"))
)

predSet_extended2_formula <- as.formula(
  paste(outcomeName, "~", paste(predSet_extended2, collapse = "+"))
)

predSet_kitchensink_formula <- as.formula(
  paste(outcomeName, "~", paste(paste0("`", predSet_kitchensink, "`"), collapse = "+"))   # Had
to alter this formula to account for variable names with hyphens
)



# Create recipes for each predictor set
# The step_bin2factor ensures outcome is a 2-level factor
recipe_bm <- recipe(predSet_bm_formula, data = trainDat) %>%
   step_bin2factor(all_of(outcomeName))

recipe_extended1 <- recipe(predSet_extended1_formula, data = trainDat) %>%
   step_bin2factor(all_of(outcomeName))

recipe_extended2 <- recipe(predSet_extended2_formula, data = trainDat) %>%
   step_bin2factor(all_of(outcomeName))

recipe_kitchensink <- recipe(predSet_kitchensink_formula, data = trainDat) %>%
   step_bin2factor(all_of(outcomeName))



# Create main formula and recipe (uses all variables)
# predSet_main_formula <- as.formula(
#    paste(outcomeName, "~", paste(".", collapse = "+"))
# )

# recipe_main <- recipe(predSet_main_formula, data = trainDat)

# Aggregate all recipes into a list
recipes <- list(predSet_bm = recipe_bm,
                predSet_extended1 = recipe_extended1,
                predSet_extended2 = recipe_extended2,
                predSet_kitchensink = recipe_kitchensink
)

# Save recipes for use in model training
```

```
saveRDS(recipes, file = paste0(params$my_path, "recipes.rds"))
# saveRDS(recipe_main, file = paste0(params$my_path, "recipe_main.rds"))
```

Step 6: Specify cross-validation parameters

Cross-validation helps ensure your models will generalize to new data. Here you will specify:

1. **Number of folds**: How many parts to split your training data into (typically 5 or 10)
2. **Stratification variable**: Whether to ensure each fold has similar outcome proportions
3. **Grid size**: How many hyperparameter combinations to test (larger = slower but more thorough)
4. **Tuning metric**: Which metric to optimize (roc_auc for balanced data, pr_auc for imbalanced)

Guidelines:

- **Folds**: 5-10 folds is standard. More folds = longer computation time
- **Stratification**: Use stratification if your outcome is imbalanced (less than 30% or more than 70% in one category)
- **Grid size**: Start with 5, increase to 10-20 if you have time and want better tuning
- **Metric**: Use roc_auc for balanced outcomes, pr_auc for imbalanced outcomes

Question 5: What cross-validation parameters did you choose? Justify your choices based on your dataset characteristics (especially outcome balance) and computational resources.

I used 5 fold validation, which I stratified with our outcome variable because of the imbalance in “y_binary”.

I set the grid size to 10 because I wanted to try and find a nice middle ground between time and analysis. I want to pick up on the subtle interactions between variables, especially in the tree based models, but also don’t want to be computationally inefficient.

Because of the significant imbalance in the data, I used pr_auc instead of roc_auc. What we are looking to predict is customers making term deposits (positive cases), and pr_auc is more sensitive to those than roc_auc.

```
# Specify number of cross-validation folds
cvFolds <- 5

# Specify stratification variable if needed
stratVar <- "y_binary"

# Specify grid size for hyperparameter tuning
gridSize <- 10

# Specify performance metric for tuning
tuneMetrics <- "pr_auc"
```

Step 7: Specify variables for fairness assessment

Algorithmic fairness is an important consideration in predictive modeling. Here you will identify variables that represent characteristics for which you want to assess whether your models perform equitably.

Important notes:

- Variables must be categorical (factor or character)
- Common fairness variables include: gender, race/ethnicity, socioeconomic status, disability status
- You can specify a multi-category variable (e.g., race with multiple levels) or binary dummy variables (e.g., race_white, race_black)
- Start with multi-category variables; if fairness issues arise, investigate with binary dummies

Question 6: Which variables did you select for fairness assessment? Why are these variables important in your context? Are there other fairness considerations you should keep in mind for your prediction problem?

The three variables I selected for fairness assessment were job, marital status, and education.

I chose these 3 because they represent key socioeconomic characteristics that could unintentionally influence the modeling process. The data does not contain direct demographic information like race and gender, but these variables could still affect the fairness of our model.

```
# Specify categorical variables for fairness assessment
equityVars <- c("job", "marital", "education")

# Verify all specified variables are categorical
equityVarsCheck <- apply(trainDat[, equityVars], 2,
                         function(x) { is.factor(x) || is.character(x) })
print(equityVarsCheck)
```

```
##       job   marital education
##      TRUE      TRUE      TRUE
```

Step 8: Review and verify specifications

Before proceeding to model training, let's review all specifications to ensure everything is correctly configured.

```
# Review prediction objective
cat("Prediction Objective:\n")
```

```
## Prediction Objective:
```

```
cat("  Outcome:", outcomeDescription, "\n")
```

```
##   Outcome: I am predicting whether or not a customer will open a term deposit account
```

```
cat("  Timepoint:", timepointDescription, "\n")
```

```
##   Timepoint: I am making predictions throughout the entire duration of the marketing campaig
n.
```

```
cat("  Population:", populationDescription, "\n\n")
```

```
##   Population: The population is all of the customers contacted as a part of a marketing campa
ign.
```

```
# Review predictor sets
cat("Predictor Sets:\n")
```

```
## Predictor Sets:
```

```
for (name in names(allPredsets)) {
  cat("  ", name, ":", length(allPredsets[[name]]), "predictors\n")
}
```

```
##    predSet_bm : 5 predictors
##    predSet_extended1 : 9 predictors
##    predSet_extended2 : 8 predictors
##    predSet_kitchensink : 44 predictors
```

```
cat("\n")
```

```
# Review modeling approaches
cat("Modeling Approaches by Predictor Set:\n")
```

```
## Modeling Approaches by Predictor Set:
```

```
for (name in names(specifiedLearners)) {
  cat("  ", name, ":", paste(specifiedLearners[[name]], collapse = ", "), "\n")
}
```

```
##    predSet_bm : glm, lasso
##    predSet_extended1 : glm, lasso, random_forest, xgboost
##    predSet_extended2 : glm, lasso, random_forest, xgboost
##    predSet_kitchensink : glm, lasso, random_forest, xgboost
```

```
cat("\n")
```

```
# Review cross-validation parameters
cat("Cross-Validation Parameters:\n")
```

```
## Cross-Validation Parameters:
```

```
cat("  Folds:", cvFolds, "\n")
```

```
##   Folds: 5
```

```
cat("  Stratification:", ifelse(is.null(stratVar), "None", stratVar), "\n")
```

```
##   Stratification: y_binary
```

```
cat("  Grid size:", gridSize, "\n")
```

```
##   Grid size: 10
```

```
cat("  Tuning metric:", tuneMetrics, "\n\n")
```

```
##   Tuning metric: pr_auc
```

```
# Review fairness variables
cat("Fairness Assessment Variables:\n")
```

```
## Fairness Assessment Variables:
```

```
cat("  ", paste(equityVars, collapse = ", "), "\n")
```

```
##    job, marital, education
```

Question 7: After reviewing all specifications above, do you feel confident your learner specifications are appropriate for your prediction problem? Are there any concerns or adjustments you might want to make before model training?

Yes, I am confident my learner specifications are appropriate for my prediction problem. I built the modeling approaches to evaluate both simple and complex relationships within the data. The models begin simple and interpretable, and increase in complexity.

Additionally, my cross validation parameters accurately represent the imbalance in the data, and balance accuracy and computational efficiency.

Step 9: Aggregate and save all specifications

Now all specifications will be aggregated into a single list object and saved for use in model training.

Note: This saves only metadata and specifications - no actual data is included in this file.

```r
# Create cross-validation parameters list
cvParams <- list(
  cvFolds = cvFolds,
  seed = seed,
  gridSize = gridSize,
  tuneMetrics = tuneMetrics,
  stratVar = stratVar
)

# Create learning objective labels data frame
learningObjectiveLabels <- data.frame(
  outcomeDescription = outcomeDescription,
  timepointDescription = timepointDescription,
  populationDescription = populationDescription
)

# Calculate positive outcome proportion from training data
obsOutcome <- as.numeric(trainDat[[outcomeName]])
positiveProportion <- sum(obsOutcome) / length(obsOutcome)

cat("Positive outcome proportion in training data:",
    round(positiveProportion, 3), "\n")
```

```
## Positive outcome proportion in training data: 0.117
```

```r
# Aggregate all specifications into learnerSpec list
learnerSpec <- list(
  my_path = params$my_path,
  trainDataName = "trainDat.rds",  # @TODO Verify this matches your Milestone 3 filename
  testDataName = "testDat.rds",    # @TODO Verify this matches your Milestone 3 filename
  learningObjectiveLabels = learningObjectiveLabels,
  outcomeName = outcomeName,
  allPredsets = allPredsets,
  specifiedLearners = specifiedLearners,
  cvParams = cvParams,
  equityVars = equityVars,
  positiveProportion = positiveProportion
)

# Save learner specifications
saveRDS(learnerSpec, file = paste0(params$my_path, "learnerSpec.rds"))

cat("\nLearner specifications saved successfully to:",
    paste0(params$my_path, "learnerSpec.rds"), "\n")
```

```
##
## Learner specifications saved successfully to: /share/dsa495601f25/tmgrover/learnerSpec.rds
```

Congratulations! You have completed Milestone 4. Your learner specifications are now saved and ready for model training in the next milestone.